

want virtualisation. For data centres, it makes sense to run multiple servers (Web, mail, etc.) on a single machine. These servers are mostly under-utilised, so clubbing them on one machine with a VM for each of the existing machines enables fewer machines, less rack space and lower electricity consumption.

For enterprises, serving users' desktops on a VM simplifies management, IT servicing, security considerations and costs, by virtue of the reduced expenditure on desktops.

For developers, testing code written for different architectures or target systems becomes easier, since access to the actual system becomes optional. For example, a new mobile phone platform can be virtualised on a developer machine rather than actually deploying the software on the phone hardware each time, allowing for the software to be developed along with the hardware. The virtualised environment can also be used as validation for the hardware platform itself, before going into production, to avoid the costs that arise later due to changes that might be needed in the hardware.

There are several such examples that can be cited for any kind of application or use-case. It's not impossible to imagine a virtualised system being beneficial anywhere a computer is used.

Now is a good time to get acquainted with some of the terms (the mandatory alphabet soup) that we'll be using throughout the article:

- VM – virtual machine
- VMM – virtual machine monitor
- Guest OS – the OS that is run within a VM
- Host OS – the OS that runs on the physical computer system and hosts guests
- Paravirtualised guest – the guest OS that is modified to have the knowledge of a VMM
- Full virtualisation – the guest OS is run unmodified in this environment
- Hypervisor – an analogous term for a VMM
- Hypercall – infrastructure, via which a paravirtualised guest and the VMM communicate

Types of VMM

There are several virtual machine monitors available. They differ in various aspects like scope, motivation, and method of implementation. A few types of monitor software are:

- 'Native' hypervisors: These VMMs have an OS associated with them. A complete software-based implementation will need a scheduler, a memory management subsystem and an IO device model to be exported to the guest OS. Examples are: VMWare ESX server, Xen, KVM, and IBM mainframes. In IBM mainframes, the VMM is an inherent part of the architecture.
- Containers: In this type of virtualisation, the guest OS and the host OS share the same kernel. Different namespaces are allocated for different guests. For example, the process identifiers, file descriptors, etc., are virtualised in the sense that a PID obtained for a process in the guest OS will only be valid within that guest. The guest can have a different

userland (for example, a different distribution) from the host. Examples are OpenVZ, FreeVPS and Linux-VServer.

- Emulation: Each and every instruction in the guest is emulated. It is possible to run code compiled for different architectures on a computer—for example, running ARM code on a PowerPC machine. Other examples are qemu and pearpc. qemu supports multiple CPU types, and it runs ARM code under x86 as well as x86 under x86, whereas pearpc only emulates the PPC platform.

Virtualisation on x86

Virtualising the x86 architecture is difficult to do since the instruction and register sets are not compatible with virtualisation. Not all accesses to privileged instructions or registers raise a trap. So we either have to emulate the guest entirely or patch it at run-time to behave in a particular way. This was true till about four years back, before virtualisation-specific instructions were added to the architecture.

With the two leading x86 processor manufacturers, Intel and AMD, adding virtualisation extensions to their processors, virtualising the x86 platform seamlessly has become easier. The ideas behind their virtualisation extensions are more or less the same, with the implementation, instructions and register sets being slightly different.

The new extensions add a new mode, the 'guest-mode', in addition to the user-mode and kernel-mode that we had (ring -1 in addition to the rings 0-3, with the hypervisor residing in ring -1). The implementations also enable support for hiding the privileged state. Disabling interrupts while in the guest mode will not affect the host-side interrupts in any way.

Open source virtualisation

Now that we've seen what virtualisation is about and what's needed on the software side to present a virtual machine to a guest operating system, let's talk about the strides open source software has been making in this field.

Xen was the first open-source hypervisor to be announced. The Xen project was started when hardware extensions to virtualisation were not yet available, and the developers took the paravirtualisation approach towards virtualising a system. The Xen team created a new hypervisor, taking bits from the Linux kernel, to run modified Linux guests. A privileged Linux guest called the Dom0, has access to the system hardware and arbitrates the access to physical resources by guest operating systems.

The Xen project got wide acceptance and was backed by a large number of companies—developers from IBM, Red Hat, Novell, Intel, AMD, all contributed to the Xen code base. It was even included in enterprise Linux offerings from various distributions as the supported virtualisation technology.

When the hardware manufacturers on x86 started adding virtualisation extensions to the processors, unmodified guests could be made to run on hypervisors. The deficiencies of the x86 instruction set were masked by these advances.

With this advancement, along came a new line of thought: why have a separate hypervisor, when all a